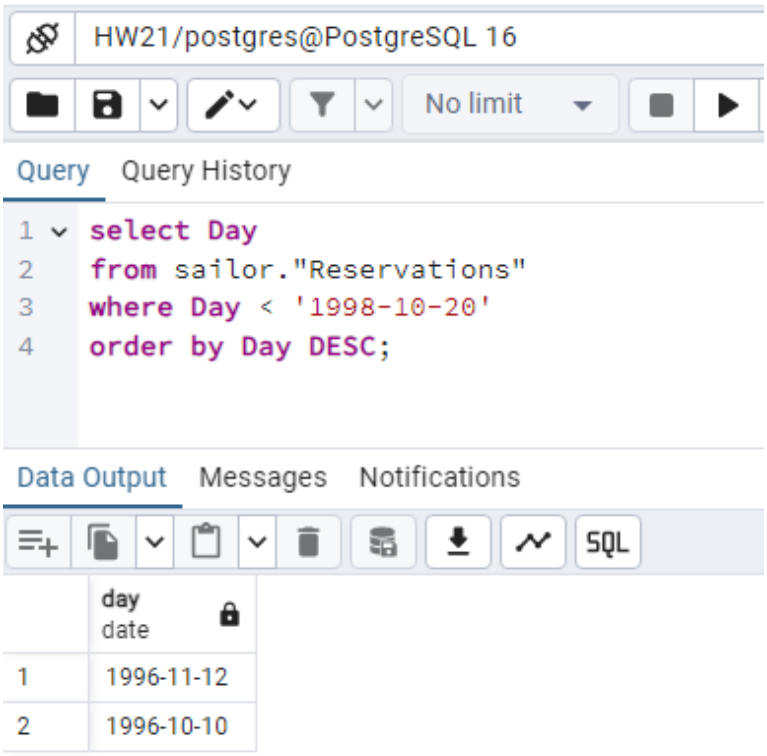
سینا ملکی- مکتب ۱۱۲ – HW21

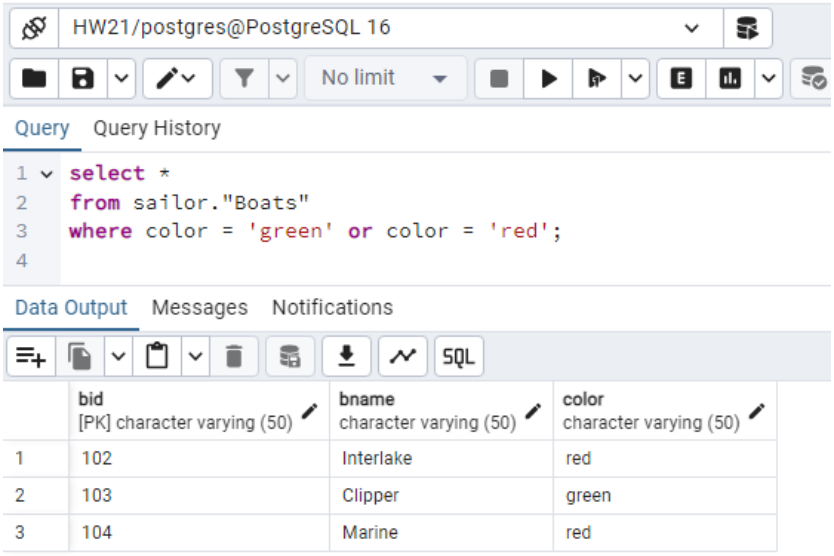
Sailors

1.1



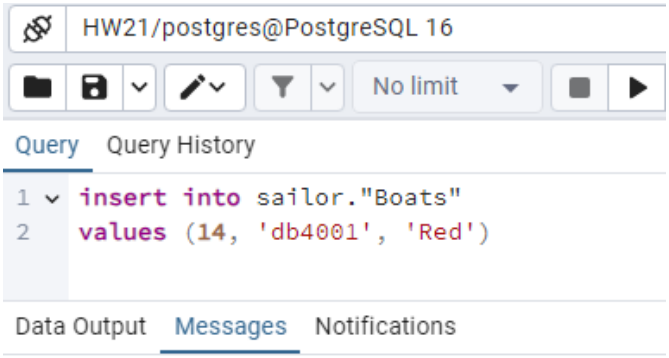
	day date
1	1996-11-12
2	1996-10-10

1.2

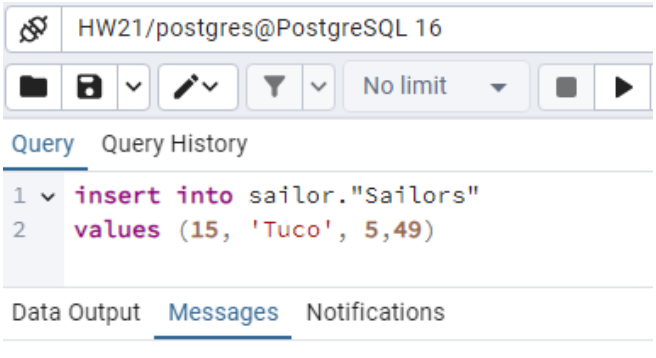


	bid [PK] character varying (50)	bname character varying (50)	color character varying (50)
1	102	Interlake	red
2	103	Clipper	green
3	104	Marine	red

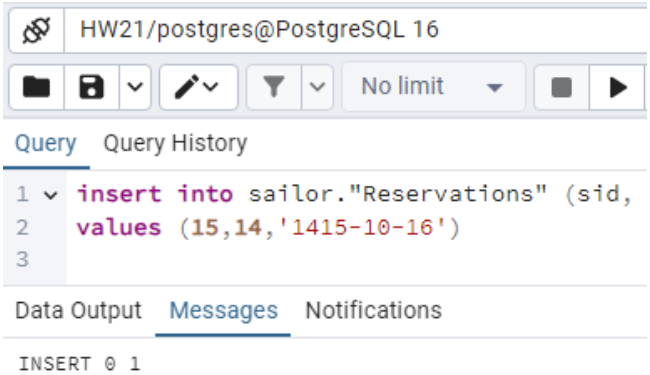
1.3


HW21/postgres@PostgreSQL 16
No limit
Query
Query History
1 insert into sailor."Boats"
2 values (14, 'db4001', 'Red')
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 92 msec.

1.4

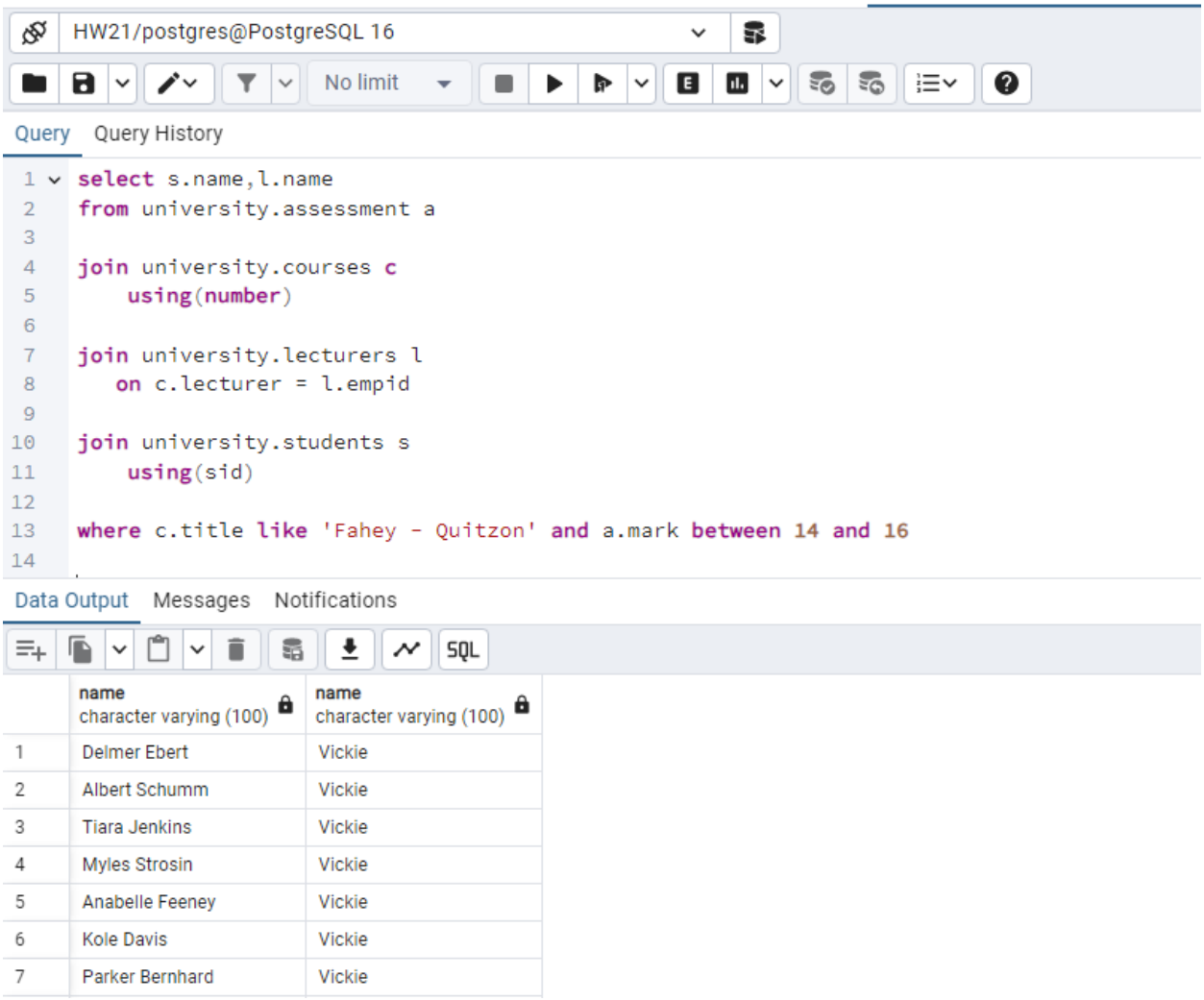

HW21/postgres@PostgreSQL 16
No limit
Query
Query History
1 insert into sailor."Sailors"
2 values (15, 'Tuco', 5,49)
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 79 msec.

1.5


HW21/postgres@PostgreSQL 16
No limit
Query
Query History
1 insert into sailor."Reservations" (sid,
2 values (15,14,'1415-10-16')
3
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 176 msec.

University

2.1



	name character varying (100)	name character varying (100)
1	Delmer Ebert	Vickie
2	Albert Schumm	Vickie
3	Tiara Jenkins	Vickie
4	Myles Strosin	Vickie
5	Anabelle Feeney	Vickie
6	Kole Davis	Vickie
7	Parker Bernhard	Vickie

2.2

HW21/postgres@PostgreSQL 16

No limit

Query | Query History

```
1  select name
2  from university.students
3  where birthdate = (select min(birthdate) from university.students)
4
5
6
```

Data Output | Messages | Notifications

SQL

	name character varying (100)
1	Malinda Mitchell

2.3

HW21/postgres@PostgreSQL 16

No limit

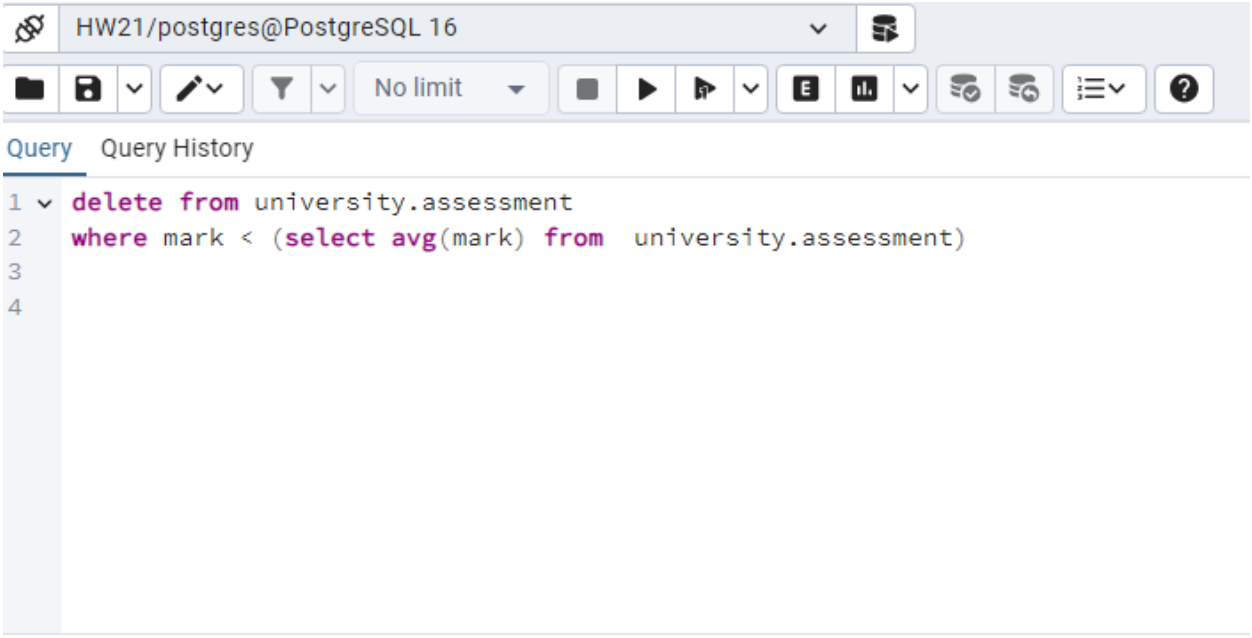
Query | Query History

```
1  select *
2  from university.assessment
3  where mark > (select AVG(mark)
4                from university.assessment)
5
```

Data Output | Messages | Notifications

SQL

	sid [PK] character varying (50)	number [PK] character varying (50)	empid character varying (50)	mark double precision
1	66354542	98579	2	15
2	40879517	85219	3	17
3	65190196	85219	4	13
4	79753200	13508	7	13
5	38862297	22929	11	13
6	5130251	71936	12	14
7	8588120	56876	14	18
8	44176308	35823	15	14
9	58936070	13011	21	13
10	73861450	87054	25	14
11	85159099	49707	26	15
12	8161587	57175	29	16


HW21/postgres@PostgreSQL 16
No limit
Query
Query History
1 delete from university.assessment
2 where mark < (select avg(mark) from university.assessment)
3
4
Data Output
Messages
Notifications
DELETE 2047
Query returned successfully in 59 msec.

2.4

HW21/postgres@PostgreSQL 16

No limit

Query | Query History

```
1  select name
2  from university.lecturers
3  where empid = (select number
4                 from university.courses
5                 where title like 'Carter Inc')
6
7
8
9
```

Data Output | Messages | Notifications

SQL

	name character varying (100)
1	Alec

HW21/postgres@PostgreSQL 16

No limit

Query | Query History

```
1  update university.lecturers
2  set name = 'Tony'
3  where empid = (select number
4                 from university.courses
5                 where title like 'Carter Inc')
6
7
8
9
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 55 msec.
```

HW21/postgres@PostgreSQL 16

No limit

Query | Query History

```
1  select name
2  from university.lecturers
3  where empid = (select number
4                 from university.courses
5                 where title like 'Carter Inc')
6
7
8
9
```

Data Output | Messages | Notifications

SQL

	name character varying (100)
1	Tony

2.5!

HW21/postgres@PostgreSQL 16

No limit

Query | Query History

```
1   -- select *
2   -- from university.courses
3   -- where title like 'Test'
4   -- insert into university.assessment
5   -- values ('66354542', '5050', '2', null)
6
7   select c.number,c.title
8   from university.courses c
9
10  join university.assessment a
11      using(number)
12
13  group by c.number,c.title
14  having count(a.mark) = 0
15
```

Data Output | Messages | Notifications

SQL

	number [PK] character varying (10)	title character varying (50)
1	5050	Test

3

HW21/postgres@PostgreSQL 16

No limit

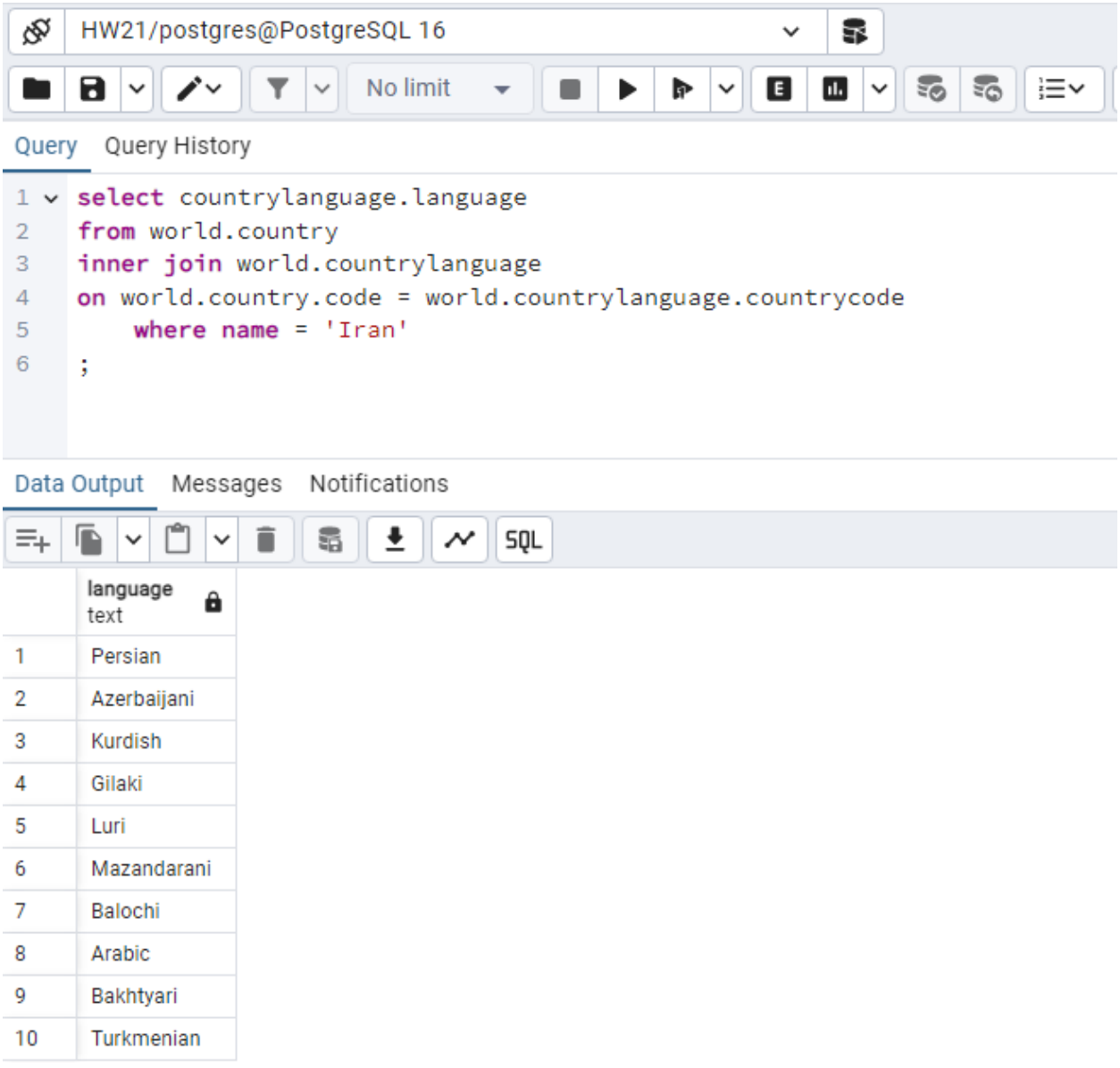
Query | Query History

```
1  select bank."borrower".customer_name
2  from bank.borrower
3  inner join bank.customer
4  on bank.customer.customer_name = bank.borrower.customer_name
5      where loan_number in (select loan_number number from bank.loan
6                            where loan.amount > 50 )
7  ;
8
```

Data Output | Messages | Notifications

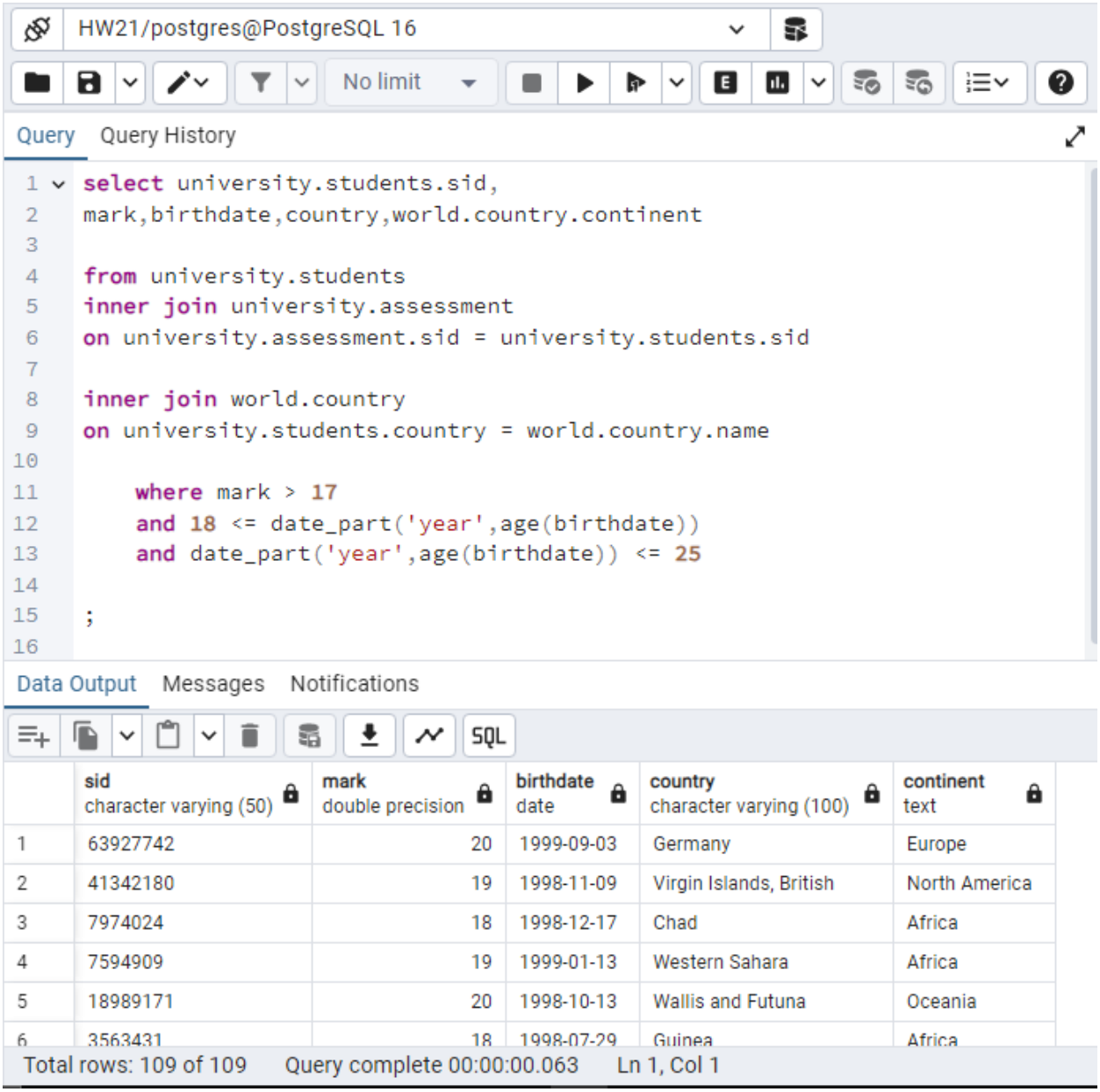
	customer_name character varying (100)
1	Ahmadi
2	Alavi
3	Hamidi
4	Hosseini
5	Kabiri
6	Moradi
7	Pahlavan
8	Rezaie

4



	language text
1	Persian
2	Azerbaijani
3	Kurdish
4	Gilaki
5	Luri
6	Mazandarani
7	Balochi
8	Arabic
9	Bakhtyari
10	Turkmenian

5(extra)



	sid character varying (50)	mark double precision	birthdate date	country character varying (100)	continent text
1	63927742	20	1999-09-03	Germany	Europe
2	41342180	19	1998-11-09	Virgin Islands, British	North America
3	7974024	18	1998-12-17	Chad	Africa
4	7594909	19	1999-01-13	Western Sahara	Africa
5	18989171	20	1998-10-13	Wallis and Futuna	Oceania
6	3563431	18	1998-07-29	Guinea	Africa

What is Selfrelation ?

Types of Relationships in DBMS

There are three primary types of relationships in DBMS. Each type of relationship plays a unique role in database design.

- One-to-One relationship
- One-to-Many or Many-to-One relationship
- Many-to-Many relationship

Self-Referencing Relationships

A self-referencing relationship is also known as recursive relationship and it is useful is cases when a table has relationship with itself. It is used for representing hierarchical data.

Example:

An “Employee” entity where each employee can have manager who is also an employee.

Source:

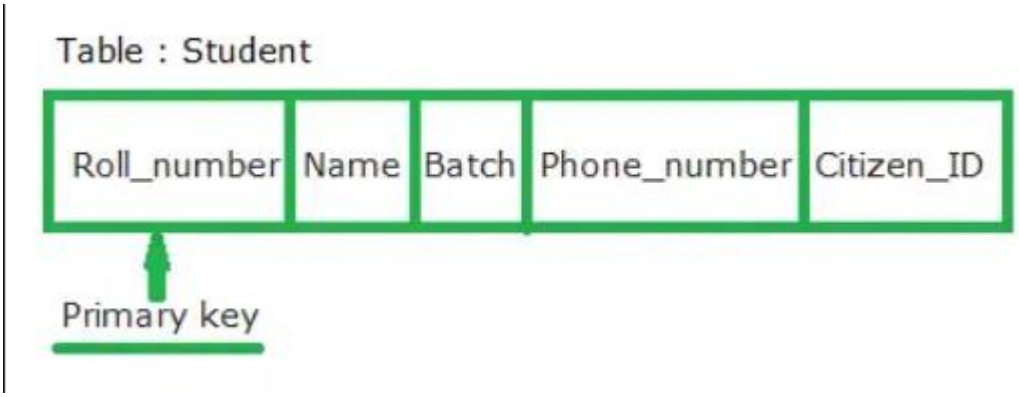
https://www.geeksforgeeks.org/what-is-relationship-type-in-dbms/

Primary Key vs Unique Key.

Primary Key

A primary key is a column of a table that uniquely identifies each tuple (row) in that table. The primary key enforces integrity constraints to the table. Only one primary key is allowed to use in a table. The primary key does not accept any duplicate and NULL values. The primary key value in a table changes very rarely so it is chosen with care where the changes can occur in a seldom manner. A primary key of one table can be referenced by the foreign key of another table.

For a better understanding of the primary key, we take a table named Student table, having attributes such as Roll_number, Name, Batch, Phone_number, and Citizen_ID.



The roll number attribute can never have an identical and NULL value, because every student enrolled in a university can have a unique roll number, therefore two students cannot have the same roll number, and each row in a table is uniquely identified with the student's roll number. So, we can make Roll_number attribute as a primary key in this case.

Features of Primary Key

Some of the essential features of Primary Keys are discussed below.

- There will be no duplicate row in case of a Primary Key.
- Only a single primary key exists for a table.
- Primary Key contains NOT NULL constraints.
- The primary Key can be made from one or more table fields.

Unique Key

Unique Key constraints also identify an individual tuple uniquely in a relation or table. A table can have more than one unique key, unlike a primary key. Unique key constraints can accept only one NULL value for the column. Unique constraints are also referenced by the foreign key of another table. It can be used when someone wants to enforce unique constraints on a column and a group of columns which is not a primary key.

For a better understanding of the unique key, we take the Student table with Roll_number, Name, Batch, Phone_number, and Citizen_ID attributes.

Roll number attribute is already assigned with the primary key and Citizen_ID can have unique constraints where each entry in a Citizen_ID column should be unique because each citizen of a country must have his or her Unique identification number like an Aadhaar Number. But if the student is migrated to another country in that case, he or she would not have any Citizen_ID and the entry could have a NULL value as only one NULL is allowed in the unique constraint.

Features of Unique Key

Some of the essential features of Unique Keys are discussed below.

- There can be more than one unique key for a table.
- Unique Keys have the liberty of having NULL values in the column.
- Unique Keys can be formed from one or more tables.
- Foreign Keys can refer to Unique Keys for referencing.

Conclusion

- The primary key will not accept NULL values whereas the Unique key can accept NULL values.
- A table can have only one primary key whereas there can be multiple unique keys on a table.
- A Clustered index is automatically created when a primary key is defined whereas a Unique key generates the non-clustered index.
- A Primary Key can be a Unique Key, but a Unique Key cannot be a primary key.

https://www.geeksforgeeks.org/difference-between-primary-key-and-unique-key/